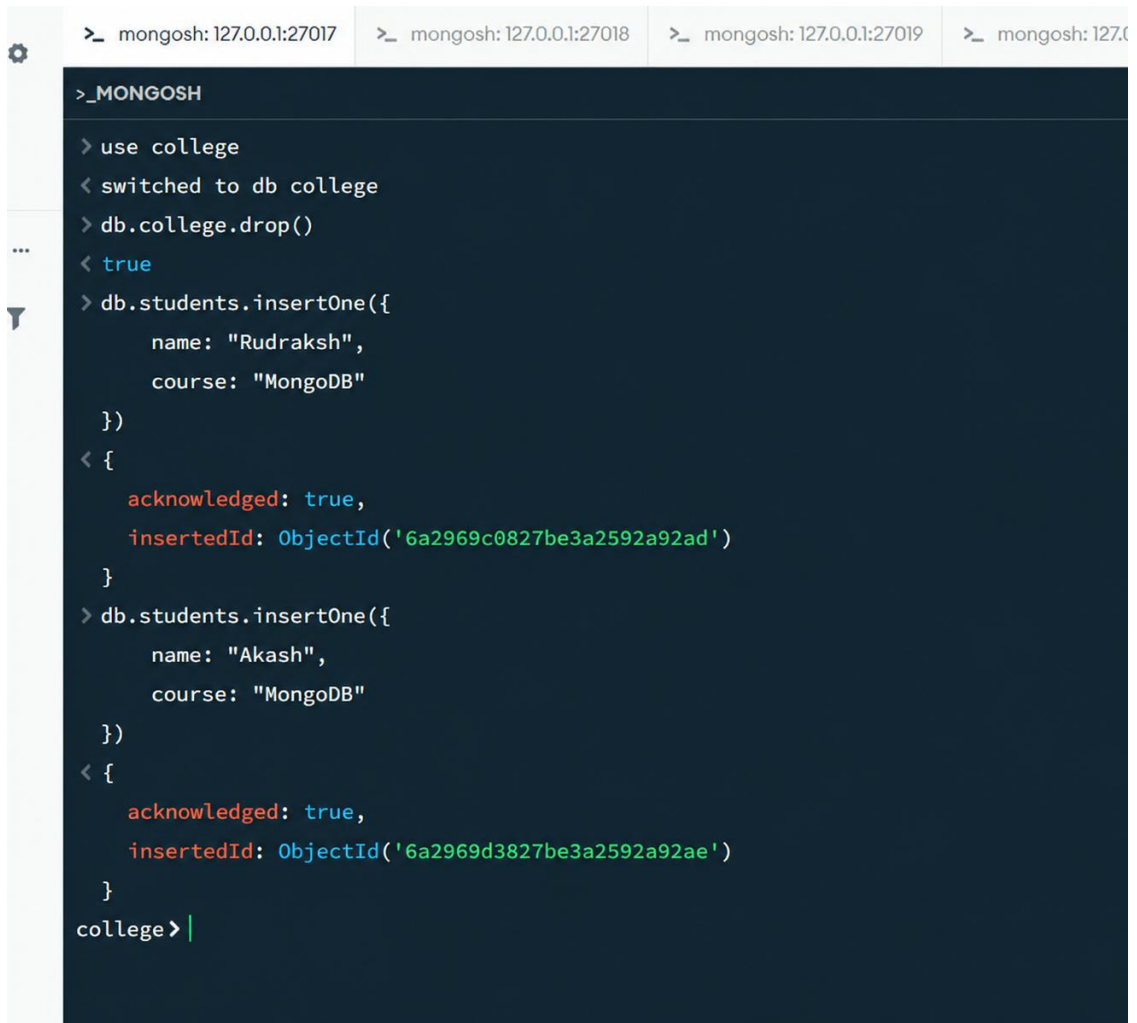


Replica-Port27017



The screenshot shows a MongoDB mongosh terminal window with four tabs at the top, all labeled 'mongosh: 127.0.0.1:27017'. The main terminal area has a dark background with light-colored text. The prompt is '>_MONGOSH'. The user enters 'use college', and the response is '< switched to db college'. Then, the user enters 'db.college.drop()', and the response is '< true'. Next, the user enters 'db.students.insertOne({ name: "Rudraksh", course: "MongoDB" })', and the response is '< { acknowledged: true, insertedId: ObjectId('6a2969c0827be3a2592a92ad') }'. Then, the user enters 'db.students.insertOne({ name: "Akash", course: "MongoDB" })', and the response is '< { acknowledged: true, insertedId: ObjectId('6a2969d3827be3a2592a92ae') }'. Finally, the prompt changes to 'college> |'.

```
>_MONGOSH
> use college
< switched to db college
> db.college.drop()
< true
> db.students.insertOne({
  name: "Rudraksh",
  course: "MongoDB"
})
< {
  acknowledged: true,
  insertedId: ObjectId('6a2969c0827be3a2592a92ad')
}
> db.students.insertOne({
  name: "Akash",
  course: "MongoDB"
})
< {
  acknowledged: true,
  insertedId: ObjectId('6a2969d3827be3a2592a92ae')
}
college> |
```

Replica-Port27018

```
>_ mongosh: 127.0.0.1:27017 >_ mongosh: 127.0.0.1:27018 >_ mongosh: 127.0.0.1:27019 >_ mongosh: 127.0.0.1:27119 >_ mongosh: 127.0.0.1:27111 +
>_MONGOSH
> use college
< switched to db college
> show collections
< students
> db.college.find()
<
> db.college.find.pretty()
❌ * TypeError: db.college.find.pretty is not a function
> db.college.find().pretty()
<
> db.students.find().pretty()
< {
  _id: ObjectId('6a296445deac5ca203efeb89'),
  name: 'Divyansh',
  course: 'MongoDB'
}
{
  _id: ObjectId('6a2969c0827be3a2592a92ad'),
  name: 'Divyansh',
  course: 'MongoDB'
}
{
  _id: ObjectId('6a2969d3827be3a2592a92ae'),
  name: 'Akash',
  course: 'MongoDB'
}
> rs.status()
< {
  set: 'myReplSet',
  date: 2026-06-10T13:46:19.386Z,
  myState: 1,
  term: Long('5'),
  syncSourceHost: '',
  syncSourceId: -1,
  heartbeatIntervalMillis: Long('2000'),
```

```
>_MONGOSH
```

```
> rs.status()
```

```
< {
  set: 'myReplSet',
  date: 2026-06-10T13:46:19.386Z,
  myState: 1,
  term: Long('5'),
  syncSourceHost: '',
  syncSourceId: -1,
  heartbeatIntervalMillis: Long('2000'),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
  optimes: {
    lastCommittedOpTime: { ts: Timestamp({ t: 1781099175, i: 1 }), t: Long('5') },
    lastCommittedWallTime: 2026-06-10T13:46:15.510Z,
    readConcernMajorityOpTime: { ts: Timestamp({ t: 1781099175, i: 1 }), t: Long('5') },
    appliedOpTime: { ts: Timestamp({ t: 1781099175, i: 1 }), t: Long('5') },
    durableOpTime: { ts: Timestamp({ t: 1781099175, i: 1 }), t: Long('5') },
    writtenOpTime: { ts: Timestamp({ t: 1781099175, i: 1 }), t: Long('5') },
    lastAppliedWallTime: 2026-06-10T13:46:15.510Z,
    lastDurableWallTime: 2026-06-10T13:46:15.510Z,
    lastWrittenWallTime: 2026-06-10T13:46:15.510Z
  },
  lastStableRecoveryTimestamp: Timestamp({ t: 1781099155, i: 1 }),
  electionCandidateMetrics: {
    lastElectionReason: 'stepUpRequestSkipDryRun',
    lastElectionDate: 2026-06-10T13:45:25.421Z,
    electionTerm: Long('5'),
    lastCommittedOpTimeAtElection: { ts: Timestamp({ t: 1781099116, i: 1 }), t: Long('4') },
    lastSeenWrittenOpTimeAtElection: { ts: Timestamp({ t: 1781099116, i: 1 }), t: Long('4') },
    lastSeenOpTimeAtElection: { ts: Timestamp({ t: 1781099116, i: 1 }), t: Long('4') },
    numVotesNeeded: 2,
    priorityAtElection: 1,
  }
}
```

```

        priorityAtElection: 1
    },
    members: [
        {
            _id: 0,
            name: '127.0.0.1:27017',
            health: 0,
            state: 0,
            stateStr: '(not reachable/healthy)',
            uptime: 0,
            optime: { ts: Timestamp({ t: 0, i: 0 }), t: Long('-1') },
            optimeDurable: { ts: Timestamp({ t: 0, i: 0 }), t: Long('-1') },
            optimeWritten: { ts: Timestamp({ t: 0, i: 0 }), t: Long('-1') },
            optimeDate: 1970-01-01T00:00:00.000Z,
            optimeDurableDate: 1970-01-01T00:00:00.000Z,
            optimeWrittenDate: 1970-01-01T00:00:00.000Z,
            lastAppliedWallTime: 2026-06-10T13:45:32.974Z,
            lastDurableWallTime: 2026-06-10T13:45:32.974Z,
            lastWrittenWallTime: 2026-06-10T13:45:32.974Z,
            lastHeartbeat: 2026-06-10T13:46:15.370Z,
            lastHeartbeatRecv: 2026-06-10T13:45:39.966Z,
            pingMs: Long('0'),
            lastHeartbeatMessage: 'Error connecting to 127.0.0.1:27017 :: caused by :: onInvoke :: caused by :: No connection could be made because the t
            syncSourceHost: '',
            syncSourceId: -1,
            infoMessage: '',
            configVersion: 1,
            configTerm: 5
        },
    ],

```

```

    },
    {
        _id: 1,
        name: '127.0.0.1:27018',
        health: 1,
        state: 1,
        stateStr: 'PRIMARY',
        uptime: 378,
        optime: { ts: Timestamp({ t: 1781099175, i: 1 }), t: Long('5') },
        optimeDate: 2026-06-10T13:46:15.000Z,
        optimeWritten: { ts: Timestamp({ t: 1781099175, i: 1 }), t: Long('5') },
        optimeWrittenDate: 2026-06-10T13:46:15.000Z,
        lastAppliedWallTime: 2026-06-10T13:46:15.510Z,
        lastDurableWallTime: 2026-06-10T13:46:15.510Z,
        lastWrittenWallTime: 2026-06-10T13:46:15.510Z,
        syncSourceHost: '',
        syncSourceId: -1,
        infoMessage: '',
        electionTime: Timestamp({ t: 1781099125, i: 1 }),
        electionDate: 2026-06-10T13:45:25.000Z,
        configVersion: 1,
        configTerm: 5,
        self: true,
        lastHeartbeatMessage: ''
    },

```

```
},
{
  _id: 2,
  name: '127.0.0.1:27019',
  health: 1,
  state: 2,
  stateStr: 'SECONDARY',
  uptime: 346,
  optime: { ts: Timestamp({ t: 1781099175, i: 1 }), t: Long('5') },
  optimeDurable: { ts: Timestamp({ t: 1781099175, i: 1 }), t: Long('5') },
  optimeWritten: { ts: Timestamp({ t: 1781099175, i: 1 }), t: Long('5') },
  optimeDate: 2026-06-10T13:46:15.000Z,
  optimeDurableDate: 2026-06-10T13:46:15.000Z,
  optimeWrittenDate: 2026-06-10T13:46:15.000Z,
  lastAppliedWallTime: 2026-06-10T13:46:15.510Z,
  lastDurableWallTime: 2026-06-10T13:46:15.510Z,
  lastWrittenWallTime: 2026-06-10T13:46:15.510Z,
  lastHeartbeat: 2026-06-10T13:46:17.697Z,
  lastHeartbeatRecv: 2026-06-10T13:46:18.304Z,
  pingMs: Long('0'),
  lastHeartbeatMessage: '',
  syncSourceHost: '127.0.0.1:27018',
  syncSourceId: 1,
  infoMessage: '',
  configVersion: 1,
  configTerm: 5
}
],
```

```

    },
    ok: 1,
    '$clusterTime': {
      clusterTime: Timestamp({ t: 1781099175, i: 1 }),
      signature: {
        hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAA=', 0),
        keyId: Long('0')
      }
    },
    operationTime: Timestamp({ t: 1781099175, i: 1 })
  }
}
> use college

db.students.insertOne({
  name: "Rahul",
  course: "Failover Test"
})
< already on db college
> db.students.insertOne({
  name: "Rahul",
  course: "Failover Test"
})
< {
  acknowledged: true,
  insertedId: ObjectId('6a296ac5306b79eda6d1ec96')
}
college> |

```

Replica-Port 27019

```
> mongosh: 127.0.0.1:27017 > mongosh: 127.0.0.1:27018 > mongosh: 127.0.0.1:27019 > mongosh: 127.0.0.1:27119 > mongosh: 127.0.0.1:27111 +
```

```
> _MONGOSH
> use college
< switched to db college
> db.students.find().pretty()
< {
  _id: ObjectId('6a2969c0827be3a2592a92ad'),
  name: 'Divyansh',
  course: 'MongoDB'
}
{
  _id: ObjectId('6a2969d3827be3a2592a92ae'),
  name: 'Akash',
  course: 'MongoDB'
}
{
  _id: ObjectId('6a296ac5306b79eda6d1ec96'),
  name: 'Rahul',
  course: 'Failover Test'
}
> rs.status()
< {
  set: 'myReplSet',
  date: 2026-06-10T13:47:52.659Z,
  myState: 2,
  term: Long('5'),
  syncSourceHost: '127.0.0.1:27018',
  syncSourceId: 1,
  heartbeatIntervalMillis: Long('2000'),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
  optimes: {
    lastCommittedOpTime: { ts: Timestamp({ t: 1781099265, i: 1 }), t: Long('5') },
    lastCommittedWallTime: 2026-06-10T13:47:45.595Z,
    readConcernMajorityOpTime: { ts: Timestamp({ t: 1781099265, i: 1 }), t: Long('5') },
    localReadMajorityOpTime: { ts: Timestamp({ t: 1781099265, i: 1 }), t: Long('5') },
    localReadWallTime: 2026-06-10T13:47:45.595Z
  }
}
```

```

members: [
  {
    _id: 0,
    name: '127.0.0.1:27017',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',
    uptime: 13,
    optime: { ts: Timestamp({ t: 1781099265, i: 1 }), t: Long('5') },
    optimeDurable: { ts: Timestamp({ t: 1781099265, i: 1 }), t: Long('5') },
    optimeWritten: { ts: Timestamp({ t: 1781099265, i: 1 }), t: Long('5') },
    optimeDate: 2026-06-10T13:47:45.000Z,
    optimeDurableDate: 2026-06-10T13:47:45.000Z,
    optimeWrittenDate: 2026-06-10T13:47:45.000Z,
    lastAppliedWallTime: 2026-06-10T13:47:45.595Z,
    lastDurableWallTime: 2026-06-10T13:47:45.595Z,
    lastWrittenWallTime: 2026-06-10T13:47:45.595Z,
    lastHeartbeat: 2026-06-10T13:47:51.434Z,
    lastHeartbeatRecv: 2026-06-10T13:47:51.917Z,
    pingMs: Long('0'),
    lastHeartbeatMessage: '',
    syncSourceHost: '127.0.0.1:27018',
    syncSourceId: 1,
    infoMessage: '',
    configVersion: 1,
    configTerm: 5
  },

```

```

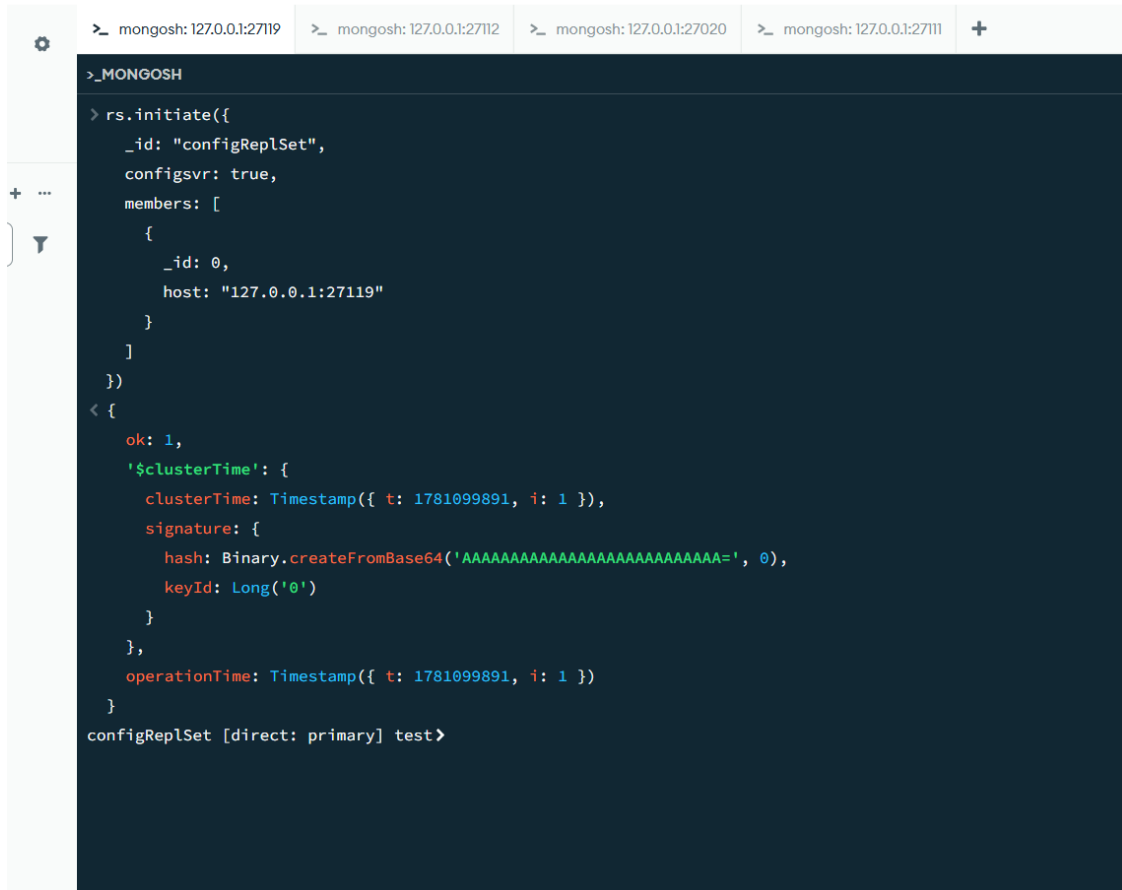
  {
    _id: 1,
    name: '127.0.0.1:27018',
    health: 1,
    state: 1,
    stateStr: 'PRIMARY',
    uptime: 439,
    optime: { ts: Timestamp({ t: 1781099265, i: 1 }), t: Long('5') },
    optimeDurable: { ts: Timestamp({ t: 1781099265, i: 1 }), t: Long('5') },
    optimeWritten: { ts: Timestamp({ t: 1781099265, i: 1 }), t: Long('5') },
    optimeDate: 2026-06-10T13:47:45.000Z,
    optimeDurableDate: 2026-06-10T13:47:45.000Z,
    optimeWrittenDate: 2026-06-10T13:47:45.000Z,
    lastAppliedWallTime: 2026-06-10T13:47:45.595Z,
    lastDurableWallTime: 2026-06-10T13:47:45.595Z,
    lastWrittenWallTime: 2026-06-10T13:47:45.595Z,
    lastHeartbeat: 2026-06-10T13:47:50.840Z,
    lastHeartbeatRecv: 2026-06-10T13:47:52.180Z,
    pingMs: Long('0'),
    lastHeartbeatMessage: '',
    syncSourceHost: '',
    syncSourceId: -1,
    infoMessage: '',
    electionTime: Timestamp({ t: 1781099125, i: 1 }),
    electionDate: 2026-06-10T13:45:25.000Z,
    configVersion: 1,
    configTerm: 5
  },

```

```
{
  _id: 2,
  name: '127.0.0.1:27019',
  health: 1,
  state: 2,
  stateStr: 'SECONDARY',
  uptime: 440,
  optime: { ts: Timestamp({ t: 1781099265, i: 1 }), t: Long('5') },
  optimeDate: 2026-06-10T13:47:45.000Z,
  optimeWritten: { ts: Timestamp({ t: 1781099265, i: 1 }), t: Long('5') },
  optimeWrittenDate: 2026-06-10T13:47:45.000Z,
  lastAppliedWallTime: 2026-06-10T13:47:45.595Z,
  lastDurableWallTime: 2026-06-10T13:47:45.595Z,
  lastWrittenWallTime: 2026-06-10T13:47:45.595Z,
  syncSourceHost: '127.0.0.1:27018',
  syncSourceId: 1,
  infoMessage: '',
  configVersion: 1,
  configTerm: 5,
  self: true,
  lastHeartbeatMessage: ''
},
ok: 1,
'$clusterTime': {
  clusterTime: Timestamp({ t: 1781099265, i: 1 }),
  signature: {
    hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
    keyId: Long('0')
  }
},
operationTime: Timestamp({ t: 1781099265, i: 1 })
}
college> |
```

Sharding:

Config Server Replica Set:



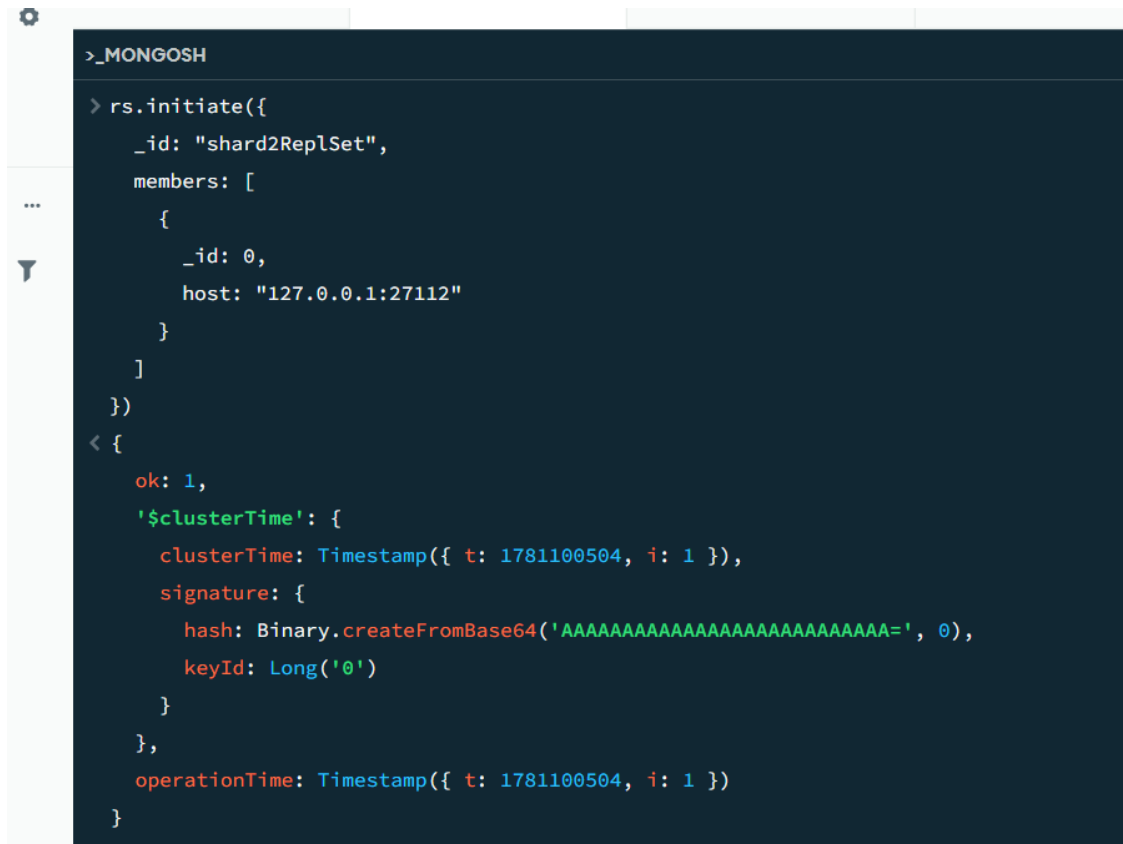
The screenshot shows a MongoDB shell interface with four tabs at the top, each representing a mongosh instance on a different port: 127.0.0.1:27119, 127.0.0.1:27112, 127.0.0.1:27020, and 127.0.0.1:27111. The active tab is the first one. The shell prompt is `>_MONGOSH`. The user has entered the command `rs.initiate({ _id: "configReplSet", configsvr: true, members: [ { _id: 0, host: "127.0.0.1:27119" } ] })`. The shell has responded with a JSON object indicating success: `{ ok: 1, '$clusterTime': { clusterTime: Timestamp({ t: 1781099891, i: 1 }), signature: { hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAA=', 0), keyId: Long('0') }, operationTime: Timestamp({ t: 1781099891, i: 1 }) } }`. Below the response, the prompt `configReplSet [direct: primary] test>` is visible.

```
>_MONGOSH
> rs.initiate({
  _id: "configReplSet",
  configsvr: true,
  members: [
    {
      _id: 0,
      host: "127.0.0.1:27119"
    }
  ]
})
< {
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1781099891, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1781099891, i: 1 })
}
configReplSet [direct: primary] test>
```

Shard 1:

```
>_MONGOSH
> rs.initiate({
  _id: "shard1ReplSet",
  members: [
    {
      _id: 0,
      host: "127.0.0.1:27111"
    }
  ]
})
< {
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1781100006, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1781100006, i: 1 })
}
shard1ReplSet [direct: primary] test>
```

Shard2:



```
>_MONGOSH

> rs.initiate({
  _id: "shard2ReplSet",
  members: [
    {
      _id: 0,
      host: "127.0.0.1:27112"
    }
  ]
})
< {
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1781100504, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1781100504, i: 1 })
}
```

mongos (Query Router)

```

> sh.addShard("shard1ReplSet/127.0.0.1:27111")
< {
  shardAdded: 'shard1ReplSet',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1781100670, i: 40 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1781100670, i: 40 })
}
> sh.addShard("shard2ReplSet/127.0.0.1:27112")
< {
  shardAdded: 'shard2ReplSet',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1781100687, i: 36 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1781100687, i: 36 })
}
> sh.status()
<
  shardingVersion

  {
    _id: ObjectId('6a296d738f421348038a1809'),
    clusterId: ObjectId('6a296d738f421348038a1808')
  }

```

shards

```
[
  {
    _id: 'shard1ReplSet',
    host: 'shard1ReplSet/127.0.0.1:27111',
    topologyTime: Timestamp({ t: 1781100670, i: 14 }),
    replSetConfigVersion: -1
  },
  {
    _id: 'shard2ReplSet',
    host: 'shard2ReplSet/127.0.0.1:27112',
    topologyTime: Timestamp({ t: 1781100687, i: 12 }),
    replSetConfigVersion: -1
  }
]
```

active mongoses

```
[
  {
    '8.3.2': 1
  }
]
```

autosplit

```
{
  'Currently enabled': 'yes'
}
```

balancer

```
{
  'Currently enabled': 'yes',
  'Currently running': 'no',
  'Failed balancer rounds in last 5 attempts': 0,
  'Migration Results for the last 24 hours': 'No recent migrations'
}
```

shardedDataDistribution

```
[]
```

databases

```
[
  {
    database: {
      _id: 'config',
      primary: 'config',
      partitioned: true
    },
    collections: {}
  }
]
```

```

>_MONGOSH
> sh.enableSharding("college")
< {
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1781100763, i: 4 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1781100763, i: 4 })
}
> use college
< switched to db college
> db.students.createIndex({ rollNo: 1 })
< rollNo_1
> sh.shardCollection("college.students", { rollNo: 1 })
< {
  collectionsharded: 'college.students',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1781100867, i: 45 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1781100867, i: 44 })
}
> for (let i = 1; i <= 20; i++) {
  db.students.insertOne({
    rollNo: i,
    name: "Student" + i,
    course: "MongoDB"
  });
}

```

```

    }
  > for (let i = 1; i <= 20; i++) {
    db.students.insertOne({
      rollNo: i,
      name: "Student" + i,
      course: "MongoDB"
    });
  }
< {
  acknowledged: true,
  insertedId: ObjectId('6a297159e2e4f03b492493ee')
}
> db.students.getShardDistribution()
<
Shard shard2ReplSet at shard2ReplSet/127.0.0.1:27112

{
  data: '1KiB',
  docs: 20,
  chunks: 1,
  'estimated data per chunk': '1KiB',
  'estimated docs per chunk': 20
}

Totals

{
  data: '1KiB',
  docs: 20,
  chunks: 1,
  'Shard shard2ReplSet': [
    '100 % data',
    '100 % docs in cluster',
    '73B avg obj size on shard'
  ]
}

```

```

>_MONGOSH
}
> for(let i = 21; i <= 5000; i++){
  db.students.insertOne({
    rollNo: i,
    name: "Student" + i
  });
}
< {
  acknowledged: true,
  insertedId: ObjectId('6a2972d8e2e4f03b4924a762')
}
> db.students.getShardDistribution()
<
Shard shard2ReplSet at shard2ReplSet/127.0.0.1:27112

{
  data: '272KiB',
  docs: 5000,
  chunks: 1,
  'estimated data per chunk': '272KiB',
  'estimated docs per chunk': 5000
}

Totals

{
  data: '272KiB',
  docs: 5000,
  chunks: 1,
  'Shard shard2ReplSet': [
    '100 % data',
    '100 % docs in cluster',
    '55B avg obj size on shard'
  ]
}
[direct: mongos] college>

```